
OTC | X

Liquidity Intelligence Radar — Swiss OTC Market

Anforderungskatalog

nach IREB ▪ Technischer Implementierungsleitfaden ▪ Quantitative Methodik

Autor: Mihael Atanasov mihaelatanasov22@gmail.com

Auftraggeber: BEKB Trading Desk, Berner Kantonalbank AG

Domäne: Schweizer OTC-Aktien — 243 ausserbörsliche Titel

Version: 2.1

Datum: Mai 2026

Klassifikation: **Proprietär & Vertraulich**

Mit Präzision gefertigt • für den Schweizer OTC-Markt

Zusammenfassung für die Geschäftsleitung

DER SCHWEIZER AUSSERBÖRSLICHE MARKT – repräsentiert durch die OTC-X-Plattform der Berner Kantonalbank – umfasst 243 kotierte Titel und ein Handelsvolumen, das sich gleichwohl einer systematischen Überwachung entzieht. Die vorliegende Plattform **OTC-X Liquidity Intelligence Radar** schliesst diese Lücke, indem sie fragmentierte Transaktionsdaten in ein signalreiches, täglich aktualisiertes Entscheidungsinstrument transformiert.

Datenbasis	139 474 Trades über 22+ Jahre, 243 Wertpapiere, 10 Sektoren
Metriken-Engine	26 Kennzahlen pro ISIN pro Handelstag inkl. Amihud-Illiquiditätsratio
Anomalie-Radar	Gewichteter Composite Score (0–7) mit sechs Severity-Stufen
Dashboard	4 Tabs, 11 interaktive Plotly-Charttypen, CSV-Export
Automatisierung	Vollautomatische Pipeline via GitHub Actions (05:00 UTC, täglich)
Testabdeckung	115 automatisierte Tests (pytest + pytest-bdd)
Live-Zugang	https://otc-x-radar.streamlit.app (öffentlich, read-only)

Tabelle 1: Dokumentenhistorie

Version	Datum	Autor	Änderung
1.0	März 2026	M. Atanasov	Initiale Erstellung (zwei separate Dokumente)
2.0	März 2026	M. Atanasov	Konsolidierung, IREB-Erweiterung, Use Cases, User Stories, Codebeispiele, TikZ-Architekturdiagramm
2.1	Mai 2026	M. Atanasov	Faktualisierung: Datenstände (Trades, Metrikzeilen, Handelshistorie), LOC-Zählung, Testabdeckung, CSS-Umfang, CI/CD-Listing auf aktuelle Pipeline-Konfiguration synchronisiert

Inhaltsverzeichnis

Zusammenfassung für die Geschäftsleitung	i
1 Projekthintergrund und Vision	1
1.1 Die Opazität des Schweizer OTC-Marktes	1
1.2 Paradigmenwechsel: Das Wertversprechen	2
1.3 Projektziele nach SMART-Kriterien	2
1.4 Kenndaten auf einen Blick	3
1.5 Erkenntnisse aus der Realisierung	3
1.6 Strategischer Ausblick	4
2 Requirements Engineering nach IREB	5
2.1 Methodisches Fundament	5
2.2 Stakeholder-Analyse	6
2.3 Systemkontext und Abgrenzung	6
2.4 Annahmen und Abhängigkeiten	6
2.5 Use Cases	7
2.6 User Stories	8
2.7 Funktionale Anforderungen	9
2.8 Nicht-funktionale Anforderungen	11
2.9 Randbedingungen (Constraints)	12
2.10 Risikoanalyse	13
2.11 Traceability Matrix	13
3 Customer Journey — Nutzererlebnis	14
3.1 Journey 1 — Das Morgenbriefing (BEKB-Händler)	14
3.2 Journey 2 — Wöchentliche Risikoprüfung (Compliance Officer)	15
3.3 Journey 3 — Sektorale Tiefenanalyse (Analyst)	15
4 Systemarchitektur und Implementierung	16
4.1 Architekturübersicht	16
4.2 Technologie-Stack	17

4.3	Polars als analytische Engine: Architekturentscheidung	18
4.4	Streamlit und die CSS-Injektionsmethodik	18
4.4.1	CSS-Injektion: Swiss Institutional Design	19
4.5	Die vierstufige ETL-Pipeline	19
4.5.1	Stufe 1 — Valor-Extraktion (<code>soft_crawl.py</code>)	20
4.5.2	Stufe 2 — Trade-Datenabholung (<code>fetcher.py</code>)	20
4.5.3	Stufe 3 — Datenkonsolidierung (<code>build_master_parquet.py</code>)	20
4.5.4	Stufe 4 — Metriken-Engine (<code>metrics.py</code>)	20
4.6	ISIN-Synthese: Der Luhn-Algorithmus	20
4.7	Pipeline-Orchestrierung und Receipt-Logging	22
4.8	Frontend-Architektur	22
4.9	CI/CD-Automatisierung	23
5	Quantitative Methodik der Metriken-Engine	24
5.1	Überblick: 26 Kennzahlen pro ISIN pro Handelstag	24
5.2	Phase 1: Tagesaggregation	24
5.3	Phase 2: Liquiditätsmetriken	25
5.3.1	Preisdynamik	25
5.3.2	Amihud-Illiquiditätsratio	25
5.3.3	Spread-Proxy nach Corwin–Schultz	26
5.4	Phase 3: Rolling Baselines (30 Handelstage)	26
5.5	Phase 4: Anomalie-Erkennung und Composite Scoring	27
5.5.1	Binäre Flag-Signale	27
5.5.2	Gewichteter Anomaly Score	27
5.5.3	Severity-Klassifikation	28
5.6	Korrelationsanalyse: Pearson-Matrix mit Dreiecksmaske	28
5.7	Boxplot-Statistik: Amihud-Illiquidität nach Sektor	29
5.8	Volatilitätsglättung: Dreistufige EWMA-Kaskade	29
5.9	Security-History: σ -Band-Konstruktion	30
5.10	3D-Markt-Explorer: Fünfdimensionale Datenprojektion	31
5.11	Volumen-Preis-Scatter: Logarithmische Skalierung	32
5.12	Off-Book-Anteil: Statistische Korrektur für Querschnittsanalysen	32
6	Dashboard-Architektur und Visualisierung	34
6.1	Designphilosophie	34
6.2	Tab-Architektur	35
6.3	11 Charttypen im Überblick	35
6.4	Volatilitätsglättung: SMA und EWMA	35
6.5	Dashboard-Screenshots	36
7	Qualitätssicherung und Engineering-Standards	39
7.1	Teststrategie	39

7.2	Code-Konventionen	39
7.3	Pfadauflösungsstrategie	40
8	Glossar und Referenzen	41
8.1	Fachbegriffe	41
8.2	Literatur und Standards	41
A	Output-Schema-Referenz	43
B	Abkürzungsverzeichnis	45
		46

Tabellenverzeichnis

1	Dokumentenhistorie	i
1.1	Paradigmenwechsel: Ohne vs. mit Liquidity Intelligence Radar	2
1.2	Projektziele – SMART-Kriterien	3
1.3	Systemkenndaten – OTC-X Liquidity Intelligence Radar	3
2.1	Stakeholder-Register nach Einfluss und Interesse	6
2.2	Systemabgrenzung — In Scope vs. Out of Scope	6
2.3	User Stories nach Rolle und Geschäftswert	8
2.4	Projektrisiken und Gegenmassnahmen	13
2.5	Traceability Matrix — Anforderungen zu Implementierungsartefakten	13
3.1	Customer Journey — Tägliches Händler-Briefing	14
3.2	Customer Journey — Compliance-Risikoprüfung	15
3.3	Customer Journey — Sektoranalyse	15
4.1	Technologie-Zuordnung nach Architekturschicht	17
4.2	CSS-Design-System — Schweizer institutionelle Ästhetik	19
4.3	Frontend-Module und ihre Verantwortlichkeiten	22
5.1	Metriken-Pipeline — 4 Berechnungsphasen	24
5.2	Anomalie-Flags: Trigger-Bedingungen und Gewichtung	27
5.3	Severity-Klassifikation nach Anomaly Score	28
5.4	Dimensionszuweisung im 3D-Explorer	31
6.1	Dashboard-Tab-Struktur und Chart-Inventar	35
6.2	Vollständiges Chart-Inventar — 11 interaktive Visualisierungstypen	35
7.1	Teststruktur nach Modul und Kategorie	39
A.1	Vollständiges Schema von <code>daily_metrics.parquet</code> (26 Spalten)	43

Abbildungsverzeichnis

4.1	Datenflussarchitektur — Von der API-Quelle bis zum Dashboard	17
6.1	Tab 1: Overview — KPI-Karten, Marktaktivität, Sektor-Treemap	36
6.2	Tab 2: Market Data — 26-Spalten-Explorer mit Suche und Export	37
6.3	Tab 3: Analytics — Korrelations-Heatmap, Amihud-Boxplots, 3D-Explorer	37
6.4	Tab 4: Anomaly Monitor — Severity-Karten und Alert-Tabelle	38

Kapitel 1

Projekthintergrund und Vision

Was nicht gemessen wird, kann nicht gesteuert werden — doch was nicht sichtbar ist, wird nicht einmal vermisst.

1.1 Die Opazität des Schweizer OTC-Marktes

DER AUSSERBÖRSLICHE HANDEL in der Schweiz – repräsentiert durch die **OTC-X**-Plattform der Berner Kantonalbank (**Berner Kantonalbank AG (BEKB)**) – stellt ein faszinierendes Paradoxon dar: obgleich rund 243 Titel mit vereinzelter, mitunter beträchtlicher Handelsaktivität dort notiert sind, existiert bis dato kein konsolidiertes Instrument, das diese fragmentierten Datenströme zu einem kohärenten Lagebild verdichtet. Die Konsequenz ist eine **strukturelle Informationsasymmetrie**, die institutionelle Akteure vor substanzielle Herausforderungen stellt.

Der Markt zeichnet sich durch fünf zentrale Charakteristika aus, die konventionelle Analysemethoden als unzulänglich entlarven:

1. **Fragmentierung** — Handelsdaten verteilen sich auf einzelne Wertpapier-Endpunkte ohne einheitliche Gesamtsicht.
2. **Illiquidität** — Zahlreiche Titel werden lediglich sporadisch gehandelt; gängige Börsenkennzahlen (VWAP, kontinuierliche Orderbücher) sind nicht anwendbar.
3. **Opazität** — Weder öffentlich verfügbare Liquiditätsbewertungen noch automatisierte Anomalie-Warnsysteme existieren.
4. **Volumen** — 243 Wertpapiere über einen Zeitraum von 22+ Jahren ergeben circa 139 000

individuelle Transaktionsdatensätze.

- 5. Heterogenität** — Die Sektoren reichen von Banken und Versicherungen über Industrie bis hin zu Immobilien, wobei jeder Sektor ein distinktives Liquiditätsprofil aufweist.

Problemstellung

Der Schweizer OTC-Markt zeichnet sich durch **fragmentierte Datenquellen**, das **Fehlen standardisierter Liquiditätsmetriken** und **unsichtbare Anomalie-Signale** aus. Konventionelle Handelsplattformen bieten keinerlei aggregierte Sicht auf marktübergreifende Muster – ein Zustand, der angesichts der verfügbaren Datenlage weder unvermeidlich noch hinnehmbar ist.

1.2 Paradigmenwechsel: Das Wertversprechen

Die nachstehende Gegenüberstellung verdeutlicht die transformative Wirkung des **Liquidity Intelligence Radar** gegenüber dem gegenwärtigen Modus Operandi. Es handelt sich hierbei nicht um eine graduelle Verbesserung, sondern um einen fundamentalen Paradigmenwechsel in der Art und Weise, wie der Schweizer OTC-Markt wahrgenommen, analysiert und navigiert wird.

Tabelle 1.1: Paradigmenwechsel: Ohne vs. mit Liquidity Intelligence Radar

Dimension	Ohne Radar	Mit Radar
Datenquelle	Ad-hoc, manuell, fehleranfällig	Systematisch, täglich, verifizierbar
Illiquiditäts-Messung	„Fühlt sich dünn an“	Quantifiziert (Amihud λ + 30d-Baseline)
Anomalie-Erkennung	Zufällig, wenn jemand hinschaut	Automatisiert, täglich, für alle 243 Titel
Risk-Quantifizierung	Geschätzt, subjektiv	Datengestützt (Spread-Allokation, Severity-Scoring)
Timing-Signale	Keine	Volumen- & Activity-Spikes (Trading-Chancen)
Langfristige Trends	Keine Sicht	Regime-Wechsel, Sektor-Entwicklung, EWMA-Volatilität
Kundenberatung	Generisch, erfahrungsbasiert	Konkrete, datengestützte Allokationsempfehlung
Operational Speed	15 Min Briefing	2 Min Briefing – alles auf einen Blick

1.3 Projektziele nach SMART-Kriterien

Die Projektziele wurden gemäss dem SMART-Rahmenwerk definiert – **S**pezifisch, **M**essbar, **A**kzeptiert, **R**ealistisch, **T**erminiert:

Tabelle 1.2: Projektziele – SMART-Kriterien

Ziel	Beschreibung & Messkriterium
Z-01	Tägliche, vollautomatisierte Akquisition sämtlicher 243 OTC-X-Wertpapierdaten bis 06:00 UTC ohne manuelles Eingreifen
Z-02	Berechnung von 26 Liquiditätskennzahlen pro ISIN pro Handelstag, einschliesslich der Amihud-Illiquiditätsratio (λ)
Z-03	Automatisierte Anomalie-Erkennung mit gewichtetem Scoring ($S \in \{0, 2, 3, 4, 5, 7\}$) und sechs Severity-Stufen
Z-04	Bereitstellung eines interaktiven Dashboards mit 4 Tabs und 11 Charttypen, Ladezeit ≤ 5 Sekunden
Z-05	Testabdeckung von ≥ 50 automatisierten Tests (aktuell: 115 Tests, pytest + pytest-bdd)
Z-06	Vollständige Dokumentation nach IREB-Standard mit Traceability Matrix und Akzeptanzkriterien

1.4 Kenndaten auf einen Blick

Tabelle 1.3: Systemkenndaten – OTC-X Liquidity Intelligence Radar

Kenngrosse	Wert
Wertpapier-Universum	243 ausserbörslich kotierte Schweizer Titel
Sektorale Abdeckung	10 Wirtschaftssektoren (Banken, Versicherungen, Industrie, Immobilien, ...)
Historische Tiefe	22+ Jahre Handelshistorie (2004–2026)
Transaktionsdatensätze	139 474 bereinigte Trades
Tägliche Metriken	26 Kennzahlen $\times$ ISIN $\times$ Handelstag = 76 357 Zeilen
Anomalie-Score-Bereich	$S \in [0, 7]$, 6 Severity-Stufen (Clean $\rightarrow$ Extreme)
Dashboard-Charttypen	11 interaktive Plotly-Visualisierungen
Automatisierungszyklus	Täglich 05:00 UTC via GitHub Actions (zero-touch)
Speichereffizienz	Parquet mit Zstandard-Kompression ($\approx 67\%$ kleiner als CSV)
Produktionscode	6 035 Zeilen Python (1 571 Backend + 2 868 Frontend + 1 596 Tests)

1.5 Erkenntnisse aus der Realisierung

Die Konzeption und Implementierung des OTC-X-Radars hat drei Einsichten von besonderer Tragweite hervorgebracht:

- (i) **Domänenwissen geht technischer Eleganz voraus.** Das Verständnis der Mikrostruktur von OTC-Märkten – weshalb die Amihud-Ratio bedeutsam ist, wie Off-Book-Trades Volumensignale verzerren, warum Kalendertage für Rolling Baselines irreführend sind – bildet das Fundament, auf dem jede architektonische Entscheidung ruht.
- (ii) **Schlichtheit ist die höchste Form der Verfeinerung.** Das vierstufige Pipeline-Design entstammt nicht einem Streben nach Minimalismus, sondern der Erkenntnis, dass jede Transformationsstufe unabhängig testbar, wiederherstellbar und erklärbar sein muss. 115 Tests validieren nicht nur Korrektheit, sondern auch die *Intention* hinter jeder Berechnung.

- (iii) **Visualisierung ist keine Dekoration, sondern Argumentation.** Die elf Charttypen im Dashboard sind nicht ornamental. Jeder beantwortet eine spezifische analytische Fragestellung – vom 3D-Explorer, der verborgene Cluster offenbart, bis zur Anomalie-Treemap, die numerische Scores in räumliche Intuition überführt.

1.6 Strategischer Ausblick

Die gegenwärtige Iteration legt das Fundament. Die Weiterentwicklung fusst auf drei strategischen Säulen:

Säule I — Prädiktive Analytik

Integration von Zeitreihenprognosen (ARIMA, Prophet, GARCH) zur vorausschauenden Liquiditätsprojektion, die es dem Trading Desk ermöglicht, Illiquiditätsereignisse zu antizipieren, bevor sie eintreten.

Säule II — Marktübergreifende Anreicherung

Einbindung von SIX-Swiss-Exchange-Referenzdaten und makroökonomischen Indikatoren (SNB-Leitzinsen, Konsumentenpreisindex), um OTC-Bewegungen im Kontext des gesamten Schweizer Finanzökosystems einzuordnen.

Säule III — Echtzeit-Streaming

Transition von täglicher Batch-Verarbeitung zu annähernd echtzeitfähiger Datenaufnahme mittels WebSocket-basierter Feeds, wodurch die Signallatenz von 24 Stunden auf wenige Minuten reduziert wird.

*Die übergeordnete Ambition besteht darin, OTC-X nicht bloss als retrospektives Analysewerkzeug zu etablieren, sondern als **vorausschauendes Entscheidungsunterstützungssystem** — ein Wächter, der den Schweizer OTC-Markt beobachtet, damit Händler es nicht mehr eigenhändig tun müssen.*

Kapitel 2

Requirements Engineering nach IREB

Die Qualität eines Systems wird nicht durch seinen Code bestimmt, sondern durch die Präzision seiner Anforderungen.

2.1 Methodisches Fundament

Das Requirements Engineering für **OTC-X** folgt dem **IREB CPRE Foundation Level**-Standard (**International Requirements Engineering Board (IREB)**, Syllabus v3.1). Die Anforderungskategorien orientieren sich an den vier Kernaktivitäten:

1. **Ermittlung** (Elicitation) — Stakeholder-Interviews, Dokumentenanalyse, API-Exploration
2. **Dokumentation** (Documentation) — Strukturierte Spezifikation mit eindeutigen IDs, Akzeptanzkriterien und Traceability
3. **Validierung** (Validation & Negotiation) — Review-Zyklen mit dem BEKB Trading Desk
4. **Verwaltung** (Management) — Versionskontrolle via Git, Traceability durch Modulzuordnung

2.2 Stakeholder-Analyse

Tabelle 2.1: Stakeholder-Register nach Einfluss und Interesse

Stakeholder	Rolle & Interesse	Einfluss	Priorität
BEKB Trading Desk	Primärer Endnutzer; benötigt tägliche Liquiditätsübersicht und Anomalie-Alerts für 243 Titel	Hoch	Kritisch
Compliance-Abteilung	Regulatorische Transparenz; Auditierbarkeit der Anomalie-Flags und Datenherkunft	Hoch	Hoch
Risikomanagement	Portfolioweite Illiquiditätsbewertung; Frühwarnung bei Extremereignissen	Mittel	Hoch
Projektautor	Architekturentscheide, Implementierung, wissenschaftliche Methodik, Qualitätssicherung	Hoch	Kritisch
IT-Infrastruktur	Pipeline-Zuverlässigkeit, Infrastrukturkosten, CI/CD-Stabilität	Mittel	Mittel
Externe Betrachter	Zugang ausschliesslich zum Live-Dashboard (read-only); kein Zugriff auf Code oder Rohdaten	Tief	Tief

2.3 Systemkontext und Abgrenzung

Tabelle 2.2: Systemabgrenzung — In Scope vs. Out of Scope

In Scope	Out of Scope
Tägliche Akquisition aller 243 OTC-X-Wertpapierdaten	Echtzeit-Streaming (geplant für Säule III)
26-Metriken-Engine mit Anomalie-Scoring	Prädiktive Modelle (ARIMA, GARCH)
4-Tab-Dashboard mit 11 Charttypen	Automatisierter Handel / Order Execution
CI/CD-Pipeline via GitHub Actions	Multi-Mandanten-Fähigkeit
115 automatisierte Tests	Integration mit Bloomberg / Reuters
CSV-Export für Compliance	Native Mobile-Applikation

2.4 Annahmen und Abhängigkeiten

- Die OTC-X **Application Programming Interface (API)** bleibt in ihrer derzeitigen Form verfügbar und liefert konsistente JSON-/CSV-Formate.
- GitHub Actions stellt zuverlässige Cron-Trigger mit `ubuntu-latest`-Runnern bereit.
- Streamlit Community Cloud gewährleistet die öffentliche Erreichbarkeit des Dashboards ohne SLA-Garantie.
- Die Autorisierung durch die **BEKB** (Schwarzenburgstrasse 160, 3001 Bern) zur Nutzung der OTC-X-Daten besteht fort.

2.5 Use Cases

Die nachfolgenden Use Cases beschreiben die zentralen Interaktionsszenarien zwischen Akteur und System, modelliert nach dem IREB-empfohlenen Textformat.

UC-01 · Tägliches Markt-Briefing abrufen

Akteur: BEKB-Händler **Priorität:** Kritisch

Vorbedingung: Pipeline-Lauf vom selben Tag erfolgreich abgeschlossen (05:00 UTC).

Standardablauf:

1. Händler öffnet das Dashboard im Browser.
2. System zeigt den Overview-Tab mit 5 KPI-Karten und Marktaktivitätschart (90 Tage).
3. Händler identifiziert auffällige Sektoren in der Treemap.
4. Händler wechselt zum Anomaly Monitor und filtert nach $\text{Severity} \geq \text{Critical}$.
5. System zeigt gefilterte Alert-Tabelle mit verlinkten ISINs.

Nachbedingung: Händler verfügt über ein konsolidiertes Lagebild des OTC-Marktes.

Alternativer Ablauf: Bei Pipeline-Fehler zeigt das Dashboard den letzten erfolgreichen Datenstand; ein visueller Indikator weist auf das Alter der Daten hin.

UC-02 · Sicherheitsspezifische Tiefenanalyse durchführen

Akteur: BEKB-Händler / Analyst **Priorität:** Hoch

Vorbedingung: Mindestens eine ISIN wurde im Anomaly Monitor identifiziert.

Standardablauf:

1. Akteur navigiert zum Market-Data-Tab.
2. Akteur sucht die betreffende ISIN im Suchfeld.
3. System filtert die Tabelle und zeigt alle 26 Metriken für den gewählten Titel.
4. Akteur öffnet die Security-History-Ansicht (σ -Band + Volumen-Subplot).
5. Akteur exportiert die gefilterte Ansicht als CSV-Datei.

Nachbedingung: Akteur besitzt eine detaillierte Analyse des betreffenden Wertpapiers sowie ein exportierbares Audit-Artefakt.

UC-03 · Sektorübergreifende Risikoanalyse erstellen**Akteur:** Compliance Officer / Risikomanagement**Priorität:** Hoch**Vorbedingung:** Zugang zum Analytics-Tab.**Standardablauf:**

1. Akteur öffnet die Korrelations-Heatmap und identifiziert unerwartete Kopplungen.
2. Akteur wechselt zum Amihud-Boxplot und vergleicht Illiquiditätsverteilungen über Sektoren.
3. Akteur nutzt den 3D-Explorer (Volumen $\times$ Amihud $\times$ Spread, Grösse: Trade-Count).
4. Ausreisser-Cluster werden dokumentiert.
5. Rolling-Volatilitätstrend bestätigt oder widerlegt den Verdacht auf Regime-Wechsel.

Nachbedingung: Risikobewertung basiert auf quantitativen, nachvollziehbaren Metriken.

2.6 User Stories

Ergänzend zu den Use Cases werden die Kernanforderungen als User Stories formuliert, um die Perspektive des Endanwenders zu schärfen:

Tabelle 2.3: User Stories nach Rolle und Geschäftswert

ID	User Story
US-01	Als Händler möchte ich jeden Morgen innert 2 Minuten einen Marktüberblick erhalten, damit ich meine Handelsaktivitäten priorisieren kann.
US-02	Als Händler möchte ich automatisch über Anomalien informiert werden , damit ich keine marktbewegenden Ereignisse verpasse.
US-03	Als Compliance Officer möchte ich Audit-fähige CSV-Exporte der Anomalie-Daten erstellen, damit ich regulatorische Nachweispflichten erfülle.
US-04	Als Risikomanager möchte ich Illiquidität sektorübergreifend vergleichen (Amihud λ), damit ich Konzentrationsrisiken frühzeitig erkenne.
US-05	Als Analyst möchte ich mehrdimensionale Zusammenhänge im 3D-Explorer untersuchen, damit ich latente Marktstrukturen identifiziere.
US-06	Als IT-Betrieb möchte ich, dass die Pipeline bei Fehler anhält und einen Receipt-Log schreibt , damit ich Ursachen schnell diagnostizieren kann.

2.7 Funktionale Anforderungen

FA-010 · Automatisierte Datenakquisition

Priorität: Muss **Quelle:** Stakeholder-Interview

Beschreibung: Das System **muss** täglich um 05:00 UTC automatisiert alle 243 Wertpapier-Metadaten und zugehörigen Handelsdaten von der OTC-X **API** abrufen. Die Ausführung erfolgt via GitHub Actions ohne manuelles Eingreifen.

Akzeptanzkriterium: securities.csv enthält ≥ 240 valide Einträge ohne doppelte ISINs.

Trace: backend/operations/soft_crawl.py

FA-020 · 4-Stufen-ETL-Pipeline

Priorität: Muss **Quelle:** Architekturentscheid

Beschreibung: Die Pipeline **muss** vier sequentielle Stufen durchlaufen: (1) Valor-Extraktion, (2) Trade-Download, (3) Parquet-Konsolidierung, (4) Metriken-Engine. Bei Fehler in einer Stufe **muss** die Pipeline sofort anhalten und einen Receipt-Log schreiben.

Akzeptanzkriterium: Receipt-Log wird auch bei Fehler generiert; keine Datenkorruption bei partiellem Lauf.

Trace: backend/pipeline.py

FA-030 · 26-Metriken-Engine

Priorität: Muss **Quelle:** Fachliche Anforderung

Beschreibung: Das System **muss** pro ISIN und Handelstag 26 Kennzahlen berechnen, darunter Preisdynamik, Log-Renditen, Amihud-Illiquiditätsratio, Spread-Proxy, Volatilität, Rolling-Baselines und einen gewichteten Anomaly-Score.

Akzeptanzkriterium: daily_metrics.parquet mit exakt 26 Spalten; Schema validiert gegen Referenzdefinition (Anhang A); Rolling-Mediane über Handelstage-Fenster berechnet.

Trace: backend/operations/metrics.py

FA-040 · Interaktives Dashboard mit 4 Tabs und 11 Charts

Priorität: Muss **Quelle:** Customer Journey

Beschreibung: Das Dashboard **muss** vier thematische Reiter bereitstellen: Overview (KPIs, Treemaps), Market Data (Explorer mit CSV-Export), Analytics (Korrelation, 3D-Explorer), Anomaly Monitor (Severity-Karten, Alert-Tabelle).

Akzeptanzkriterium: Dashboard lädt innert 5 Sekunden; alle 11 Charttypen rendern fehlerfrei; Sektorfilter, Suche und CSV-Export funktional.

Trace: frontend/app.py, frontend/operations/charts.py

FA-050 · Anomalie-Erkennung mit Severity-Stufen**Priorität:** Muss **Quelle:** Fachliche Anforderung**Beschreibung:** Das System **muss** drei binäre Flags (Volume Spike, Activity Spike, Price Gap) zu einem gewichteten Score $S = 3 \cdot \mathbb{K}_{\text{vol}} + 2 \cdot \mathbb{K}_{\text{act}} + 2 \cdot \mathbb{K}_{\text{gap}}$ aggregieren und sechs Severity-Stufen zuordnen.**Akzeptanzkriterium:** Scores korrekt berechnet; $S \in \{0, 2, 3, 4, 5, 7\}$; Severity-Mapping konsistent mit Dashboard-Anzeige.**Trace:** backend/operations/metrics.py :: compute_anomaly_flags()**FA-060 · Security Deep-Dive mit σ -Band****Priorität:** Soll **Quelle:** Customer Journey**Beschreibung:** Für jede gewählte ISIN **soll** das Dashboard einen historischen Kursverlauf mit 30-Tage- σ -Band ($\pm 1\sigma$) und einem Volumen-Subplot anzeigen.**Akzeptanzkriterium:** σ -Band korrekt aus 30-Tage-Rolling-Standardabweichung berechnet; Band als schattierte Fläche gerendert.**Trace:** frontend/operations/charts.py :: chart_security_history()**FA-070 · CSV-Datenexport****Priorität:** Soll **Quelle:** Customer Journey (Phase 6)**Beschreibung:** Das Dashboard **soll** einen Download-Button bereitstellen, der die aktuelle Filteransicht als CSV-Datei exportiert, kompatibel mit Microsoft Excel und internen BEKB-Systemen.**Akzeptanzkriterium:** Export-Datei enthält alle sichtbaren Spalten mit korrekter Formatierung.**Trace:** frontend/app.py (Market-Data-Tab)**FA-080 · 3D-Markt-Explorer****Priorität:** Kann **Quelle:** Analysebedarf**Beschreibung:** Der Analytics-Tab **kann** einen fünfdimensionalen interaktiven 3D-Scatter-Plot bereitstellen, bei dem der Nutzer aus 12 Metrikooptionen für X-, Y-, Z-Achse, Farbe und Grösse wählen kann.**Akzeptanzkriterium:** Alle fünf Dimensions-Selektoren funktional; $\log_{10}$ -Transformation umschaltbar; Ausreisserentfernung (1.–99. Perzentil) korrekt.**Trace:** frontend/app.py (Analytics-Tab, 3D-Explorer-Sektion)

2.8 Nicht-funktionale Anforderungen

NFA-010 · Performance

Priorität: Muss

Die vollständige 4-Stufen-Pipeline **muss** innert 5 Minuten auf Standard-CI-Hardware (GitHub Actions, 2 vCPU, 7 GB RAM) abschliessen. Dashboard-Ladezeit **soll** < 5 Sekunden bei Erstaufruf betragen.

Metrik: Pipeline Wall-Clock-Time ≤ 300 s; Streamlit Initial Render ≤ 5 s.

Trace: Pipeline-Receipt-Logs (backend/logs/pipeline_receipt_*.log)

NFA-020 · Zuverlässigkeit und Wiederherstellbarkeit

Priorität: Muss

Jede Pipeline-Stufe **muss** unabhängig wiederherstellbar sein. Ein Fehler in einer Stufe **muss** die nachfolgenden Stufen stoppen, einen diagnostischen Receipt-Log erzeugen und alle zuvor abgeschlossenen Artefakte erhalten.

Trace: backend/pipeline.py :: _run_stage()

NFA-030 · Datensouveränität

Priorität: Muss

Sämtliche Daten **müssen** bis zur formellen Übergabe in der Hoheit des Autors verbleiben. Kein Dritter darf Datenströme replizieren oder abfangen.

Trace: Proprietary Notice (README)

NFA-040 · Testabdeckung

Priorität: Muss

Die Testsuite **muss** ≥ 50 automatisierte Tests umfassen (aktuell: 115 Tests, pytest + pytest-bdd). Testabdeckung erstreckt sich über Pfadauflösung, Backend-Operationen, Frontend-Utilities und Chart-Rendering.

Trace: tests/test_backend.py, tests/test_frontend.py, tests/test_paths.py

NFA-050 · Datenintegrität und Kompression**Priorität:** Muss

Alle persistenten Datenartefakte **müssen** im Apache-Parquet-Format mit Snappy- oder Zstandard-Kompression gespeichert werden.

Metrik: Parquet-Dateien ohne Schema-Override lesbar; Kompressionsrate $\geq 3\times$ gegenüber CSV.

Trace: backend/operations/build_master_parquet.py, backend/operations/metrics.py

NFA-060 · Wartbarkeit und Dokumentation**Priorität:** Soll

Sämtliche Python-Module **sollen** mit Type Hints (PEP 484) und NumPy-Style-Docstrings dokumentiert sein.

Metrik: 100 % der öffentlichen Funktionen annotiert; Docstrings auf allen 12 Python-Modulen.

Trace: Alle *.py-Dateien

NFA-070 · Swiss Institutional Design**Priorität:** Soll

Das Dashboard **soll** eine visuelle Identität im Schweizer Institutional-Stil aufweisen: Primärfarbe #B22222 (Swiss Red), serifenlose Typographie (Inter), monochrome Akzentstruktur auf weißem Hintergrund.

Trace: frontend/operations/styles.py, frontend/operations/config.py

2.9 Randbedingungen (Constraints)

C-01 · Datensouveränität und Autorisierung

Sämtliche Datenverarbeitung erfolgt im Rahmen der durch die **BEKB** (Schwarzenburgstrasse 160, 3001 Bern) erteilten Autorisierung. Die Datensouveränität verbleibt beim Projektautor bis zur formellen Übergabe. Keinerlei Datenexfiltration an Dritte gestattet.

C-02 · Technologie-Stack

Das System ist auf folgenden Stack begrenzt: Python 3.10+, Polars (**Extract, Transform, Load (ETL)**), Pandas/NumPy (Analytik), Streamlit/Plotly (Dashboard), Apache Parquet (Persistenz), GitHub Actions (**Continuous Integration (CI)/Continuous Deployment (CD)**), pytest (Testing). Keine zusätzlichen Laufzeitabhängigkeiten ohne explizite Genehmigung.

C-03 · Deployment-Modell

Das Dashboard **muss** via Streamlit Community Cloud deploybar sein. Die Backend-Pipeline **muss** innerhalb von GitHub-Actions-Runnern ausführbar sein. Keine proprietäre Cloud-Infrastruktur erforderlich.

2.10 Risikoanalyse

Tabelle 2.4: Projektrisiken und Gegenmassnahmen

ID	Risiko	Schwere	Gegenmassnahme
R-01	API-Änderung oder -Abschaltung durch OTC-X	Hoch	Retry-Logik, modulares Fetcher-Design, Receipt-Log bei Fehler
R-02	GitHub-Actions-Ausfall verhindert Pipeline	Mittel	Manueller Workflow-Dispatch als Fallback
R-03	Streamlit Community Cloud Downtime	Tief	Daten im Repo gesichert; lokale Ausführung möglich
R-04	Datenqualitätsprobleme in OTC-X-Exports	Mittel	Schema-Normalisierung, Deduplication, Null-Guards
R-05	Schwellenwertkalibrierung ungenau	Mittel	30-Tage-Median als adaptiver Baseline; Gewichte konfigurierbar

2.11 Traceability Matrix

Tabelle 2.5: Traceability Matrix — Anforderungen zu Implementierungsartefakten

ID	Priorität	Modul	Verifikation
FA-010	Muss	soft_crawl.py	test_backend.py — Wertpapieranzahl-Assertion
FA-020	Muss	pipeline.py	test_system.py — Pipeline-Ausführung
FA-030	Muss	metrics.py	test_backend.py — Metriken-Berechnungstests
FA-040	Muss	app.py, charts.py	test_frontend.py — Chart-Smoke-Tests
FA-050	Muss	metrics.py	test_backend.py — Anomaly-Score-Tests
FA-060	Soll	charts.py	test_frontend.py — Chart-Render-Tests
FA-070	Soll	app.py	Manuelle Akzeptanz
FA-080	Kann	app.py	Manuelle Akzeptanz
NFA-010	Muss	pipeline.py	Receipt-Log-Zeitanalyse
NFA-020	Muss	pipeline.py	Fehlerinjektionstests
NFA-030	Muss	Rechtsdokumentation	Proprietary Notice (README)
NFA-040	Muss	tests/	pytest-Ausführung (≥ 115 bestanden)
NFA-050	Muss	build_master_parquet.py	Parquet-Schema-Validierung
C-01	—	Rechtsdokumentation	Proprietary Notice
C-02	—	requirements.txt	Dependency-Manifest-Review
C-03	—	.github/workflows/	CI-Workflow-Validierung

Kapitel 3

Customer Journey — Nutzererlebnis

Der Wert eines Systems bemisst sich nicht an seiner Komplexität, sondern an der Geschwindigkeit, mit der es Einsicht gewährt.

Die nachfolgenden Customer Journeys bilden die primären Interaktionspfade durch OTC-X ab, modelliert nach dem Fünf-Phasen-Rahmenwerk (Awareness → Consideration → Decision → Action → Retention), adaptiert aus ISO 9241-210 für menschenzentriertes Design.

3.1 Journey 1 — Das Morgenbriefing (BEKB-Händler)

Tabelle 3.1: Customer Journey — Tägliches Händler-Briefing

Phase	Aktivität	Tab
Awareness	Händler öffnet das Dashboard um 08:00; KPI-Karten zeigen sofort den Puls des gestrigen Marktes: Gesamtvolumen, aktive ISINs, höchster Anomaly Score	Overview
Consideration	Sektor-Treemap offenbart ungewöhnliche Volumenkonzentration im Bankensektor; Top-Movers heben eine 12 %-Preislücke bei einem Mid-Cap-Industrietitel hervor	Overview
Decision	Händler navigiert zum Anomaly Monitor; filtert „Severe“-Stufe; identifiziert 3 Titel mit $\text{Score} \geq 5$	Anomaly
Action	Klick auf Market Data; Untersuchung des historischen σ -Band-Charts der markierten ISIN; CSV-Export für Compliance-Review	Market Data
Retention	Pipeline läuft am nächsten Tag automatisch um 05:00 UTC; frische Daten erwarten den Händler ohne manuelles Zutun	Automatisiert

3.2 Journey 2 — Wöchentliche Risikoprüfung (Compliance Officer)

Tabelle 3.2: Customer Journey — Compliance-Risikoprüfung

Phase	Aktivität	Tab
Awareness	Wöchentlicher Review; Officer öffnet den Analytics-Tab zur systemischen Risikoeinschätzung	Analytics
Consideration	Korrelations-Heatmap offenbart unerwartete Kopplung zwischen Volumen und Amihud-Illiquidität im Versicherungssektor	Analytics
Decision	Amihud-Boxplot bestätigt erhöhte Illiquidität; Rolling-Volatilitätstrend zeigt steigendes σ über 30 Tage	Analytics
Action	3D-Explorer zur Kartierung Volumen $\times$ Amihud $\times$ Spread; Ausreisser-Cluster identifiziert und dokumentiert	Analytics
Retention	Erkenntnisse dokumentiert; nächste Woche dient als Benchmark gegen diese Baselines	Retention

3.3 Journey 3 — Sektorale Tiefenanalyse (Analyst)

Tabelle 3.3: Customer Journey — Sektoranalyse

Phase	Aktivität	Tab
Awareness	Analyst wählt „Immobilien“ im Sektorfilter; sämtliche Metriken rekalisieren sich auf das Segment	Analytics
Consideration	Volumen-vs-Preis-Scatter zeigt: Immobilientitel clustern im Quadranten niedriges Volumen / hoher Kurs	Analytics
Decision	Vergleich mit Bankensektor; Korrelations-Heatmap um <code>spread_log_h1</code> erweitert	Analytics
Action	5 Immobilien-ISINs mit persistent hohem Amihud $\lambda > 50$ identifiziert; Export für Portfolio-Review	Market Data
Retention	Monatlicher Sektorbericht aus exportierten Daten; Dashboard als Lesezeichen für wiederkehrenden Zugriff	Retention

Kapitel 4

Systemarchitektur und Implementierung

Architektur ist gefrorene Entscheidung — jede Schicht spiegelt eine Abwägung zwischen Einfachheit und Ausdruckskraft.

4.1 Architekturübersicht

Die Architektur von OTC-X folgt dem Prinzip einer **vierstufigen ETL-Pipeline** mit strikter Separation of Concerns zwischen Backend (Polars-basiert) und Frontend (Pandas/Streamlit). Das nachstehende Diagramm illustriert den Datenfluss von der Quelle bis zur Visualisierung:

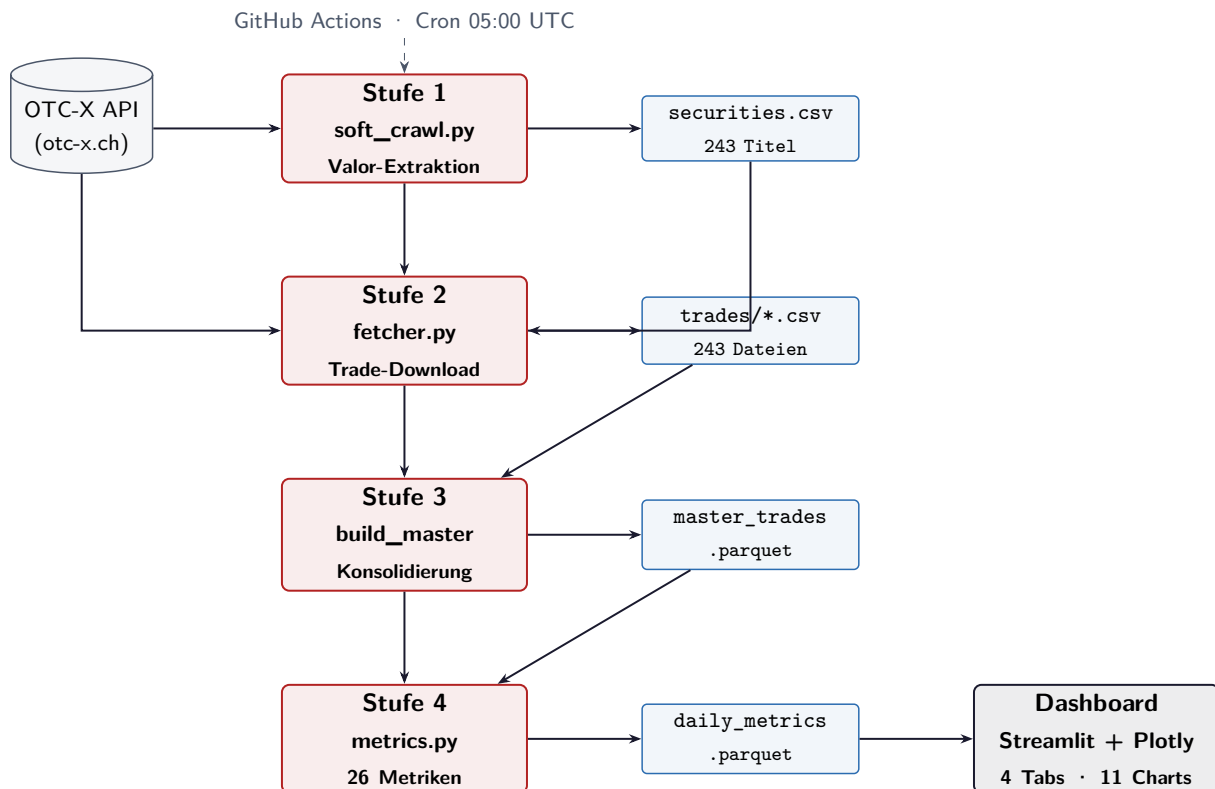


Abbildung 4.1: Datenflussarchitektur — Von der API-Quelle bis zum Dashboard

4.2 Technologie-Stack

Tabelle 4.1: Technologie-Zuordnung nach Architekturschicht

Schicht	Technologie	Begründung
Runtime	Python 3.10+	Wissenschaftliches Ökosystem, Polars/Pandas-Kompatibilität, breite Community
ETL-Engine	Polars (Rust)	Multithreaded Arrow-backed Engine; $\approx 10\times$ schneller als Pandas für Aggregationen
Analytik	Pandas + NumPy	Kompatibilität mit Streamlit und Plotly; bewährte numerische Bibliotheken
Dashboard	Streamlit + Plotly	Rapid Prototyping; 11 interaktive Charttypen; CSS-Injektion für Swiss-Design
Persistenz	Apache Parquet	Komprimierte Spaltenformate (Zstandard/Snappy); $3\times$ kleiner als CSV
CI/CD	GitHub Actions	Tägliche Cron-Ausführung (05:00 UTC); zero-touch Automation
Testing	pytest + pytest-bdd	115 Unit-, Integrations- und BDD-Akzeptanztests
Versionskontrolle	Git + GitHub	Vollständiger Audit-Trail, Branch-Protection, automatisierte Commits

4.3 Polars als analytische Engine: Architekturentscheidung

Die Wahl von **Polars** als zentrale **ETL**-Engine stellt eine bewusste Architekturentscheidung dar, die gegenüber dem verbreiteten Pandas-Ökosystem signifikante Vorteile in Durchsatz, Speichereffizienz und Ausdruckskraft bietet. Polars ist eine in **Rust** geschriebene DataFrame-Bibliothek, die über Python-Bindings zugänglich ist und auf dem **Apache Arrow**-Speicherformat operiert.

Technische Architektur von Polars

- **Zero-Copy-Speichermodell:** Polars nutzt das Apache-Arrow-Columnar-Format, das spaltenweise Speicherung ohne Serialisierungs-overhead ermöglicht. Daten werden in zusammenhängenden Speicherblöcken abgelegt, was L1/L2-Cache-Nutzung maximiert.
- **Multithreaded Query Engine:** Im Gegensatz zu Pandas, das auf dem Python-GIL (Global Interpreter Lock) operiert und somit single-threaded ist, führt Polars Operationen parallel über alle verfügbaren CPU-Kerne aus — ohne dass der Anwender explizites Threading implementieren muss.
- **Lazy Evaluation:** Polars unterstützt ein *Lazy-API*, bei dem Abfragen zunächst als logischer Plan konstruiert und anschliessend durch einen Query Optimizer reorganisiert werden — einschliesslich Predicate Pushdown, Projection Pushdown und Common Subexpression Elimination.
- **SIMD-Vektorisierung:** Arithmetische Operationen werden auf Maschinenebene mittels SIMD-Instruktionen (Single Instruction, Multiple Data) beschleunigt, was insbesondere bei den 76 357 Zeilen der Metriken-Engine messbare Performanzgewinne erbringt.

Weshalb nicht Pandas? Für die Metriken-Berechnung — insbesondere die Rolling-Window-Operationen mit `.rolling_median(window_size=30).over(İsin"` über 76 357 Zeilen — erreicht Polars einen Durchsatz, der Pandas um eine Grössenordnung ($\approx 10\times$) übertrifft. Die `.over()`-Semantik erlaubt partitionierte Fensterberechnungen in einer einzigen, deklarativen Expression, wohingegen Pandas ein explizites `groupby().apply()` erfordert, das die GIL-Limitation verschärft.

Weshalb nicht eine relationale Datenbank? Das Datenvolumen von OTC-X (139 474 Trades, 76 357 Metrikzeilen) liegt im Sweet Spot für In-Memory-Analytik. Eine PostgreSQL- oder SQLite-Instanz würde unnötige Infrastrukturkomplexität einführen, ohne bei dieser Grössenordnung einen Vorteil in Abfragegeschwindigkeit zu bieten. Polars mit Parquet-Persistenz vereint die Flexibilität einer DataFrame-API mit der Effizienz spaltenorientierter Speicherung.

4.4 Streamlit und die CSS-Injektionsmethodik

Streamlit wurde als Dashboard-Framework gewählt, da es den kürzesten Weg vom analytischen Python-Backend zur interaktiven Web-Visualisierung bietet — ohne die Notwendigkeit, eine separate Frontend-Codebasis in JavaScript zu unterhalten. Das Framework realisiert ein *reactive data flow*-Modell: jede Benutzerinteraktion löst einen vollständigen Skript-Rerun aus, wobei

`@st.cache_data(ttl=3600)` sicherstellt, dass die Parquet-Daten nur einmal pro Stunde von der Festplatte gelesen werden.

4.4.1 CSS-Injektion: Swiss Institutional Design

Die visuelle Identität des Dashboards wird durch eine **448 Zeilen umfassende CSS-Injektion** realisiert, die via `st.markdown()` mit `unsafe_allow_html=True` in den Streamlit-Rendering-Kontext eingebettet wird. Diese Methodik erlaubt die vollständige Übersteuerung der Streamlit-Standard-Widgets bei gleichzeitiger Wahrung der Framework-Integrität.

Listing 4.1: CSS-Injektionsmuster (vereinfacht)

```
1 def inject_css() -> None:
2     """Injiziert 448 Zeilen Swiss-Design-CSS in Streamlit."""
3     st.markdown(f"<style>{CSS_CONTENT}</style>",
4                 unsafe_allow_html=True)
```

Die CSS-Architektur umfasst folgende Designprinzipien:

Tabelle 4.2: CSS-Design-System — Schweizer institutionelle Ästhetik

Aspekt	Implementierung
Typographie	Inter (300–700) für Text; IBM Plex Mono für numerische Werte — Schweizer Präzision durch monospaced Zahlenalignment
Farbdisziplin	Monochromatisches Fundament (Navy auf Weiss); #B22222 als einziger Akzent; semantische Farben (Grün/Rot) ausschliesslich für Sentiment
KPI-Karten	3px border-top in Swiss Red; box-shadow: 0 1px 2px; IBM Plex Mono für Metriken bei 1.65 rem
Severity-Badges	6 Farbstufen (Clean → Extreme); Uppercase, 0.06 em Letter-Spacing, 700 Weight — maximale Scannbarkeit
Animationen	Einzig ein Pulse-Effekt (2s Zyklus, 35 % Opazität) für den Live-Status-Indikator — funktionaler Minimalismus
Hover-Interaktion	<code>translateY(-1px)</code> mit Schattenintensivierung; 0.15s ease-Transition — subtile Tiefenillusion

*Die CSS-Injektion umfasst 50+ Selektorguppen, die Streamlits BaseWeb-Widgets via Attributselektoren (`[data-baseweb]`, `[data-testid]`) gezielt übersteuern — ein Vorgehen, das **produktionsreife Kontrolle** über die visuelle Konsistenz gewährleistet, ohne die Framework-Architektur zu kompromittieren.*

4.5 Die vierstufige ETL-Pipeline

4.5.1 Stufe 1 — Valor-Extraktion (`soft_crawl.py`)

Die Pipeline beginnt mit einem nicht-invasiven Crawl der OTC-X-Plattform zur Entdeckung des vollständigen Wertpapier-Universums. Für jedes Wertpapier werden die Attribute **International Securities Identification Number (ISIN)**, Valor-Nummer, Unternehmensname und Sektorklassifikation extrahiert. Die Ausgabedatei `securities.csv` dient als autoritatives Register für alle nachgelagerten Stufen.

4.5.2 Stufe 2 — Trade-Datenabholung (`fetcher.py`)

Für jede ISIN im Wertpapierregister ruft der Fetcher die vollständige Handelshistorie von der OTC-X **API** ab. Die Implementierung umfasst:

- **ISIN-Validierung** mittels ISO 6166-Prüfzifferalgorithmus (Luhn)
- **Rate Limiting** (1 Request/Sekunde) zur Vermeidung von API-Drosselung
- **Retry-Logik** mit automatischem Neuversuch bei HTTP 403/429 nach 5 s Backoff
- **Deduplizierung** stiller Übersprungung doppelter Valor-Einträge

4.5.3 Stufe 3 — Datenkonsolidierung (`build_master_parquet.py`)

Die Konsolidierungsstufe vereint 243 individuelle Trade-CSVs zu einem einzigen, typsicheren Master-Dataset:

1. **Schema-Normalisierung** — Sämtliche Dateien mit `infer_schema_length=0` gelesen (alle Spalten als UTF8)
2. **Typecasting** — Schweizer Zahlenformate (Apostroph als Tausendertrenner) in IEEE 754 Float64 konvertiert
3. **Deduplizierung** — Exakte Zeilenduplikate eliminiert; $\approx 139\,000$ bereinigte Datensätze
4. **Kompression** — Zstandard-Kompression reduziert die Dateigrösse um $\approx 67\%$

4.5.4 Stufe 4 — Metriken-Engine (`metrics.py`)

Das quantitative Herzstück von OTC-X transformiert das Master-Trade-Dataset in ein 26-spaltiges analytisches Layer. Diese Stufe wird in Kapitel 5 eingehend behandelt.

4.6 ISIN-Synthese: Der Luhn-Algorithmus

Die Konvertierung von Schweizer Valor-Nummern zu international gültigen ISINs erfolgt algorithmisch mittels des **Luhn-Prüfzifferverfahrens** (ISO 7812):

ISIN-Synthese: Formale Definition

Gegeben eine Valor-Nummer v , wird die ISIN wie folgt konstruiert:

1. Padding: $v_{\text{pad}} = \text{zfill}(v, 9)$ auf 9 Ziffern
2. Präfixierung: $\text{prefix} = \text{CH} \| v_{\text{pad}}$ (11 Zeichen)
3. Buchstaben-Expansion: Jeder Buchstabe c wird zu $\text{ord}(c) - 55$ expandiert
4. Prüfziffer nach Luhn:

$$d_{\text{check}} = (10 - S \bmod 10) \bmod 10$$

wobei:

$$S = \sum_{i=0}^{n-1} \sigma(d_i \cdot w_i), \quad w_i = \begin{cases} 2 & \text{falls } i \equiv 0 \pmod{2} \\ 1 & \text{sonst} \end{cases}$$

und $\sigma(x) = \lfloor x/10 \rfloor + (x \bmod 10)$ die Quersumme darstellt.

Ergebnis: $\text{ISIN} = \text{CH} \| v_{\text{pad}} \| d_{\text{check}}$ (12 Zeichen)

Die nachstehende Implementierung in Python illustriert die konkrete Umsetzung:

Listing 4.2: ISIN-Berechnung mit Luhn-Prüfziffer (fetcher.py)

```

1 def calculate_isin_check_digit(isin_without_check: str) -> str:
2     """Berechnet die Luhn-Pruefziffer fuer eine 11-stellige ISIN-
3         Basis.
4
5         Beispiel: Valor 1629001 -> CH001629001 -> Pruefziffer 9
6         -> Finale ISIN: CH0016290019
7
8         """
9     digits: list[int] = []
10    for char in isin_without_check:
11        if char.isdigit():
12            digits.append(int(char))
13        else:
14            val = ord(char.upper()) - 55 # A=10, B=11, ..., Z=35
15            digits.extend(divmod(val, 10) if val >= 10 else (val,))
16
17    total = 0
18    for i, d in enumerate(reversed(digits)):
19        weight = 2 if i % 2 == 0 else 1
20        product = d * weight
21        total += (product // 10) + (product % 10)
22
23    return str((10 - total % 10) % 10)

```

4.7 Pipeline-Orchestrierung und Receipt-Logging

Der Orchestrator (`pipeline.py`) setzt eine eigens entwickelte `_TeeStream`-Klasse ein, die sämtliche Standardausgaben sowohl an die Konsole als auch an einen internen Puffer spiegelt. Nach Abschluss wird ein umfassender Receipt assembliert:

Listing 4.3: Vereinfachtes Pipeline-Orchestrierungsmuster (`pipeline.py`)

```

1 STAGES = [
2     ("Valor-Extraktion",    soft_crawl.run),
3     ("Trade-Download",     fetcher.run),
4     ("Konsolidierung",     build_master_parquet.run),
5     ("Metriken-Engine",    metrics.run),
6 ]
7
8 for name, func in STAGES:
9     log.info(f"=== {name} ===")
10    t0 = time.perf_counter()
11    try:
12        func()
13        elapsed = time.perf_counter() - t0
14        log.info(f"{name}: OK ({elapsed:.1f}s)")
15    except Exception as exc:
16        log.error(f"{name}: FEHLER - {exc}")
17        write_receipt(partial=True)
18        raise SystemExit(1)
19
20 write_receipt(partial=False)

```

4.8 Frontend-Architektur

Das Streamlit-Dashboard besteht aus sechs zweckgebundenen Modulen:

Tabelle 4.3: Frontend-Module und ihre Verantwortlichkeiten

Modul	Verantwortlichkeit
<code>config.py</code>	Markenkonstanten, Farbpaletten, Metrik-Anzeigenamen, Anomalie-Mappings
<code>styles.py</code>	CSS-Injektion (448 Zeilen) — Swiss-Precision-Theming mit #B22222-Akzent
<code>utils.py</code>	CHF-Formatierung, Prozent-Einfärbung, Score-Badge-Rendering, NaN-Sicherheit
<code>data_loader.py</code>	Gecachter Parquet → Pandas Loader mit <code>@st.cache_data(ttl=3600)</code>
<code>charts.py</code>	11 Plotly-Chart-Factories (Treemaps, Heatmaps, 3D-Scatter, σ -Bänder)
<code>components.py</code>	Header-Rendering, KPI-Karten-Assembly, Tabellen-Formatter

4.9 CI/CD-Automatisierung

Die GitHub-Actions-Pipeline (automation_pipeline.yml) führt den vollständigen ETL-Zyklus täglich aus:

Listing 4.4: CI/CD-Ablauf (vereinfacht)

```
1 # Trigger: Cron 0 5 * * * (täglich 05:00 UTC) + manuell
2 name: OTC-X Daily Pipeline
3 on:
4   schedule:
5     - cron: '0 5 * * *'
6   workflow_dispatch:
7
8 permissions:
9   contents: write      # erforderlich fuer commit-back der Pipeline-
                        Artefakte
10
11 jobs:
12   run-pipeline:
13     runs-on: ubuntu-latest
14     timeout-minutes: 30
15     steps:
16       - uses: actions/checkout@v4
17       - uses: actions/setup-python@v5
18         with: { python-version: '3.14', cache: 'pip' }
19       - run: pip install -r requirements-dev.txt      # incl. pytest-
                        bdd
20       - run: pytest tests/ -x -q                    # Test-Gate vor
                        Daten-Refresh
21       - run: python -m backend.pipeline              # 4-Stufen-ETL
22       - run: |
23         git config --global user.name 'github-actions[bot]'
24         git config --global user.email 'github-actions[bot]@users.
                        noreply.github.com'
25         git add backend/data/ backend/logs/
26         git commit -m "Automated daily OTC-X data update" \
27           || echo "No changes to commit"
28         git push
```

Kapitel 5

Quantitative Methodik der Metriken-Engine

Messen heisst Wissen — doch erst die richtige Metrik macht aus Daten Erkenntnis.

5.1 Überblick: 26 Kennzahlen pro ISIN pro Handelstag

Die Metriken-Engine transformiert rohe Transaktionsdaten in ein **26-spaltiges** analytisches Dataset. Jede Zeile repräsentiert eine eindeutige Kombination aus ISIN und Handelstag. Die Berechnung gliedert sich in vier aufeinanderfolgende Phasen:

Tabelle 5.1: Metriken-Pipeline — 4 Berechnungsphasen

Phase	Bezeichnung	Output-Spalten
1	Tagesaggregation	trades_today, volume_today_units/chf, price_min/max/first/last, volatility_daily, trade_duration_min, off_book_pct
2	Liquiditätsmetriken	price_change_pct, log_returns, spread_log_hl, amihud_daily
3	Rolling Baselines	trades/volume/volatility/amihud_30d_median
4	Anomalie-Detektion	volume_spike, activity_spike, price_gap, anomaly_score

5.2 Phase 1: Tagesaggregation

Für jedes Wertpapier i und jeden Handelstag t werden die Rohtrades zu folgenden Aggregaten verdichtet:

Handelsvolumen in CHF

$$V_{i,t}^{\text{CHF}} = \sum_{k=1}^{n_{i,t}} q_k \cdot P_k$$

wobei q_k die gehandelte Stückzahl und P_k der Kurs des k -ten Trades sind.

Intraday-Volatilität

$$\sigma_{i,t} = \sqrt{\frac{1}{n_{i,t} - 1} \sum_{k=1}^{n_{i,t}} (P_k - \bar{P}_{i,t})^2} \quad \text{mit} \quad \bar{P}_{i,t} = \frac{1}{n_{i,t}} \sum_{k=1}^{n_{i,t}} P_k$$

Für Tage mit nur einem Trade gilt konventionsgemäss $\sigma_{i,t} = 0$.

Maximale Handelsdauer (Trade Duration)

Bei zeitlich sortierten Trades mit Zeitstempeln $\tau_1, \dots, \tau_n$ (in Minuten seit Mitternacht):

$$D_{i,t} = \max_{k \in \{2, \dots, n\}} (\tau_k - \tau_{k-1})$$

Diese Kennzahl erfasst die maximale Lücke zwischen aufeinanderfolgenden Transaktionen.

5.3 Phase 2: Liquiditätsmetriken

5.3.1 Preisdynamik

Relative Preisveränderung

$$\Delta P_{i,t} \% = \frac{P_{i,t}^{\text{last}} - P_{i,t}^{\text{first}}}{P_{i,t}^{\text{first}}} \times 100$$

Stetige (logarithmische) Rendite

$$r_{i,t} = \ln \left(\frac{P_{i,t}^{\text{last}}}{P_{i,t}^{\text{first}}} \right)$$

Log-Renditen besitzen gegenüber diskreten Renditen die Vorteile der **Zeitadditivität** ($r_{[t_1, t_n]} = \sum r_{t_k}$) und des **symmetrischen Verhaltens** bei Kursgewinnen und -verlusten.

5.3.2 Amihud-Illiquiditätsratio

Die von Yakov Amihud (2002) eingeführte Kennzahl misst den **Preisimpakt pro gehandelter Volumeneinheit** und bildet das zentrale Mass für Illiquidität:

Amihud Illiquidity Ratio λ

$$\lambda_{i,t}^{\text{Amihud}} = \frac{|r_{i,t}|}{V_{i,t}^{\text{CHF}}} \times 10^6$$

Interpretation:

- Ein **hoher** λ -Wert indiziert, dass selbst geringe Handelsvolumina zu substanziellen Preisbewegungen führen — das Kennzeichen eines illiquiden Marktes.
- Der Skalierungsfaktor 10^6 normalisiert auf *Preisimpakt pro CHF-Million*.
- Schutzklausel: $V_{i,t}^{\text{CHF}} = 0 \Rightarrow \lambda_{i,t} = \text{null}$.

Literaturverweis: Amihud, Y. (2002). *Illiquidity and stock returns*. Journal of Financial Markets, 5(1), 31–56.

Die Polars-Implementierung dieser zentralen Metrik:

Listing 5.1: Amihud-Ratio in Polars (metrics.py)

```

1 df = df.with_columns(
2     pl.when(pl.col("volume_today_chf") > 0)
3         .then(
4             pl.col("log_returns").abs()
5             / pl.col("volume_today_chf")
6             * 1_000_000
7         )
8     .otherwise(None)
9     .alias("amihud_daily")
10 )

```

5.3.3 Spread-Proxy nach Corwin–Schultz**High-Low Spread Proxy**

$$S_{i,t}^{\text{HL}} = \ln \left(\frac{P_{i,t}^{\text{high}}}{P_{i,t}^{\text{low}}} \right)$$

Diese Metrik dient als Approximation der Geld-Brief-Spanne, basierend auf Corwin & Schultz (2012).

5.4 Phase 3: Rolling Baselines (30 Handelstage)

Um kontextsensitive Anomalie-Erkennung zu ermöglichen, berechnet die Engine **30-Handelstage-Rolling-Mediane** für vier Schlüsselmetriken:

Rolling Median über 30 Handelstage

Für eine Metrik x des Titels i :

$$\tilde{x}_{i,t}^{(30)} = \text{median}(x_{i,t-29}, x_{i,t-28}, \dots, x_{i,t})$$

Angewandt auf: $x \in \{\text{trades_today}, V^{\text{CHF}}, \sigma, \lambda^{\text{Amihud}}\}$

Design-Entscheidung: Der Median (anstelle des arithmetischen Mittels) wurde gewählt, da er **robust gegenüber Ausreißern** ist — eine unerlässliche Eigenschaft in einem Markt, in dem einzelne Grossblockhandel die Verteilung drastisch verzerren können.

Wertpapiere mit weniger als 30 Handelstagen erhalten dennoch einen Baseline-Wert ($\text{min_samples}=1$).

5.5 Phase 4: Anomalie-Erkennung und Composite Scoring

5.5.1 Binäre Flag-Signale

Drei unabhängige Triggerbedingungen werden evaluiert:

Tabelle 5.2: Anomalie-Flags: Trigger-Bedingungen und Gewichtung

Flag	Trigger-Bedingung	Gewicht
Volume Spike	$V_{i,t}^{\text{CHF}} > 1,5 \times \tilde{V}_{i,t}^{(30)}$	3
Activity Spike	$n_{i,t} > 1,5 \times \tilde{n}_{i,t}^{(30)}$	2
Price Gap	$ \Delta P_{i,t}\% > 5$	2

5.5.2 Gewichteter Anomaly Score

Composite Anomaly Score

$$S_{i,t} = 3 \cdot \mathbb{K}_{[V > 1,5 \tilde{V}_{30}]} + 2 \cdot \mathbb{K}_{[n > 1,5 \tilde{n}_{30}]} + 2 \cdot \mathbb{K}_{[|\Delta P\%| > 5]}$$

wobei $\mathbb{K}_{[\cdot]}$ die Indikatorfunktion bezeichnet.

Werteraum: $S \in \{0, 2, 3, 4, 5, 7\}$ (nicht alle Werte in $[0, 7]$ sind erreichbar)

Die Polars-Implementierung des Scorings:

Listing 5.2: Anomalie-Scoring in Polars (metrics.py)

```

1 df = df.with_columns([
2     (pl.col("volume_today_chf") > pl.col("volume_30d_median") * 1.5)
3     .fill_null(False).alias("volume_spike"),
4     (pl.col("trades_today").cast(pl.Float64)
5     > pl.col("trades_30d_median") * 1.5)
6     .fill_null(False).alias("activity_spike"),

```

```

7      (pl.col("price_change_pct").abs() > 5.0)
8      .fill_null(False).alias("price_gap"),
9  ])
10
11 df = df.with_columns(
12     (pl.col("volume_spike").cast(pl.UInt8) * 3
13      + pl.col("activity_spike").cast(pl.UInt8) * 2
14      + pl.col("price_gap").cast(pl.UInt8) * 2)
15     .alias("anomaly_score")
16 )

```

5.5.3 Severity-Klassifikation

Tabelle 5.3: Severity-Klassifikation nach Anomaly Score

Score	Stufe	Interpretation	Farbe
0	Clean	Kein Signal; normales Handelsverhalten	#28A745
2	Alert	Einzelner Trigger aktiv; Beobachtung empfohlen	#FD7E14
3	Alert	Volume Spike allein; erhöhte Aufmerksamkeit	#FD7E14
4	Critical	Zwei Flags aktiv; unmittelbare Prüfung erforderlich	#DC3545
5	Severe	Starke Abweichung auf mehreren Dimensionen	#7D1128
7	Extreme	Alle drei Trigger gleichzeitig aktiv	#4A0010

Design-Rationale der Gewichtung

Die asymmetrische Gewichtung ($w_{\text{vol}} = 3$ vs. $w_{\text{act}} = w_{\text{gap}} = 2$) reflektiert die domänenspezifische Erkenntnis, dass **Volumenanomalien** im OTC-Segment die stärksten Vorboten für marktbewegende Ereignisse sind. In einem Markt mit inhärent geringer Liquidität signalisiert ein plötzlicher Volumenanstieg mit hoher Wahrscheinlichkeit einen fundamentalen Informationsschock.

5.6 Korrelationsanalyse: Pearson-Matrix mit Dreiecksmaske

Die im Analytics-Tab dargestellte Korrelations-Heatmap berechnet die paarweise Pearson-Korrelation über bis zu neun Metriken und visualisiert ausschliesslich das **untere Dreieck** der symmetrischen Matrix:

Pearson-Korrelationskoeffizient

$$\rho(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad \rho \in [-1, +1]$$

Die obere Dreiecksmaske wird mittels `np.triu(np.ones(n, n), k=1)` erzeugt und über `corr.mask()` auf die Matrix angewandt, sodass nur $\frac{n(n-1)}{2}$ Koeffizienten angezeigt werden.

Analysierte Metriken: $\Delta P\%$, σ , λ^{Amihud} , V^{CHF} , Trades, Off-Book %, Anomaly Score, S^{HL} , $\ln(r)$

Farbskala: Divergierender Gradient von ■ Blau ($\rho = -1$) über Weiss ($\rho = 0$) zu ■ Rot ($\rho = +1$)

5.7 Boxplot-Statistik: Amihud-Illiquidität nach Sektor

Die sektorale Verteilung der Amihud-Ratio wird als Boxplot mit statistischer Überlagerung dargestellt. Dabei werden zwei Bereinigungsstufen vorgeschaltet:

Boxplot-Konstruktion mit Ausreisserbereinigung

Stufe 1 — Positiv-Filter: Nur Beobachtungen mit $\lambda_{i,t} > 0$ werden einbezogen (Tage ohne Preisbewegung oder Volumen liefern $\lambda = 0$ oder null).

Stufe 2 — 95. Perzentil-Cap:

$$\lambda_{\text{cap}} = Q_{0.95}(\lambda) \quad \Rightarrow \quad \lambda_{i,t}^* = \lambda_{i,t} \quad \text{für} \quad \lambda_{i,t} \leq \lambda_{\text{cap}}$$

Dies eliminiert extreme Ausreisser, die die Skalierung verzerren würden, und bewahrt gleichzeitig 95 % der Verteilung.

Boxplot-Elemente:

- **Box:** Q_1 bis Q_3 (Interquartilsabstand, IQR)
- **Median:** Q_2 (Zentralwert)
- **Whiskers:** $Q_1 - 1,5 \cdot \text{IQR}$ bis $Q_3 + 1,5 \cdot \text{IQR}$
- **Overlay:** `boxmean="sd"` überlagert $\bar{\lambda}$ und $\pm 1\sigma$ als Diamant-Marker
- **Sortierung:** Sektoren aufsteigend nach Median- λ (liquidester Sektor zuerst)

5.8 Volatilitätsglättung: Dreistufige EWMA-Kaskade

Die sektorale Volatilitätsentwicklung wird durch eine **dreistufige Glättungskaskade** aufbereitet, die kurzfristiges Rauschen eliminiert und strukturelle Volatilitätsregime hervorhebt:

Macro-Smoothing-Pipeline: SMA → EWMA → EWMA

Eingabe: Tägliche Sektorvolatilität $\bar{\sigma}_{\text{Sektor},t} = \frac{1}{|\mathcal{I}_s|} \sum_{i \in \mathcal{I}_s} \sigma_{i,t}$ (arithmetisches Mittel aller ISINs im Sektor s)

Stufe 1 — SMA-90 (Eliminierung täglicher/wöchentlicher Zyklen):

$$\sigma_t^{(1)} = \frac{1}{90} \sum_{k=0}^{89} \bar{\sigma}_{t-k} \quad (\text{min_periods} = 1)$$

Stufe 2 — EWM mit span = 120 (Verdichtung auf breite Strukturtrends):

$$\sigma_t^{(2)} = \alpha_{120} \sigma_t^{(1)} + (1 - \alpha_{120}) \sigma_{t-1}^{(2)}, \quad \alpha_{120} = \frac{2}{120 + 1} \approx 0,0165$$

Stufe 3 — EWM mit span = 60 (Entfernung residualer Oszillationen):

$$\sigma_t^{(3)} = \alpha_{60} \sigma_t^{(2)} + (1 - \alpha_{60}) \sigma_{t-1}^{(3)}, \quad \alpha_{60} = \frac{2}{60 + 1} \approx 0,0328$$

Interpretation: Die resultierende Serie $\sigma_t^{(3)}$ repräsentiert das *langfristige Volatilitätsregime* eines Sektors. Ein nachhaltiger Anstieg von $\sigma^{(3)}$ über mehrere Wochen signalisiert einen fundamentalen Regime-Wechsel — nicht bloss tägliches Rauschen. Der alternative Raw-Modus zeigt lediglich einen SMA-30 ohne Kaskade.

Listing 5.3: Dreistufige EWMA-Kaskade in Pandas (charts.py)

```
1 # Stufe 1: Simple Moving Average (90 Tage)
2 smoothed = sector_vol.rolling(90, min_periods=1).mean()
3 # Stufe 2: Exponential Weighted Mean (span=120)
4 smoothed = smoothed.ewm(span=120, adjust=False).mean()
5 # Stufe 3: Exponential Weighted Mean (span=60)
6 smoothed = smoothed.ewm(span=60, adjust=False).mean()
```

5.9 Security-History: σ -Band-Konstruktion

Für den Einzeltitel-Deep-Dive wird ein **Rolling- σ -Band** um den Kursverlauf gelegt, das die historische Streuung als visuellen Korridor abbildet:

Rolling Standard Deviation Band

Gegeben die Schlusskursreihe P_t^{last} eines Wertpapiers:

$$\hat{\sigma}_t^{(30)} = \sqrt{\frac{1}{29} \sum_{k=0}^{29} \left(P_{t-k}^{\text{last}} - \bar{P}_t^{(30)} \right)^2} \quad (\text{min_periods} = 5)$$

Obere Bande: $U_t = P_t^{\text{last}} + \hat{\sigma}_t^{(30)}$

Untere Bande: $L_t = P_t^{\text{last}} - \hat{\sigma}_t^{(30)}$

Die Forderung `min_periods=5` stellt sicher, dass das Band erst ab dem fünften Handelstag erscheint, da die Standardabweichung mit weniger als fünf Beobachtungen statistisch nicht belastbar wäre. NaN-Werte werden mit 0 aufgefüllt, um Darstellungslücken zu vermeiden. Das Band wird als semitransparente Fläche (`rgba(178,34,34,0.10)`) gerendert, während der Kursverlauf selbst als Linie in Swiss Red darübergelegt wird. Ein separates Volumen-Subplot darunter kontextualisiert die Preisbewegung.

5.10 3D-Markt-Explorer: Fünfdimensionale Datenprojektion

Der 3D-Explorer ermöglicht die simultane Visualisierung von **fünf Variablen** in einem interaktiven Scatter-Plot:

Tabelle 5.4: Dimensionszuweisung im 3D-Explorer

Dimension	Kanal	Beschreibung
X-Achse	Räumlich	Wählbar aus 12 Metriken (z. B. V^{CHF})
Y-Achse	Räumlich	Wählbar aus 12 Metriken (z. B. $\Delta P\%$)
Z-Achse	Räumlich	Wählbar aus 12 Metriken (z. B. λ^{Amihud})
Farbe	Chromatisch	Kontinuierliche Skala (<code>RdYlGn_r</code> , umgekehrt)
Grösse	Morphologisch	Min-Max-Normierung auf <code>[6, 35]</code> Pixel

Transformationen im 3D-Explorer**Logarithmische Transformation** (optional, räumliche Achsen):

$$x' = \log_{10}(x + s), \quad s = \begin{cases} |x_{\min}| + 1 & \text{falls } x_{\min} \leq 0 \\ 0 & \text{sonst} \end{cases}$$

Der Shift-Parameter s gewährleistet, dass auch negative oder Null-Werte logarithmierbar sind.

Ausreisserbereinigung (optional, alle 5 Dimensionen):

$$x_i^* = x_i \quad \text{für} \quad Q_{0.01} \leq x_i \leq Q_{0.99}$$

Beobachtungen ausserhalb des 1.–99. Perzentils werden eliminiert, um die räumliche Auflösung im Kernbereich der Verteilung zu maximieren.

Grössennormierung:

$$r_i = 6 + 29 \cdot \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Lineare Min-Max-Skalierung auf den Bereich [6, 35] Pixel Markerradius.

5.11 Volumen-Preis-Scatter: Logarithmische Skalierung

Der Scatter-Plot im Analytics-Tab projiziert jedes Wertpapier in einen zweidimensionalen Raum aus Handelsvolumen (logarithmisch) und Preisveränderung (linear), wobei die Sektorzugehörigkeit farbkodiert ist. Zwei unabhängige Perzentilfilter gewährleisten die Darstellungsqualität:

Duale Perzentilbereinigung**Volumenachse** (X, logarithmisch):

$$V_i \in [Q_{0.01}(V), Q_{0.99}(V)] \quad \text{mit} \quad V_i > 0 \quad (\text{Voraussetzung für } \log_{10})$$

Preisachse (Y, linear):

$$\Delta P_i \in [Q_{0.005}(\Delta P), Q_{0.995}(\Delta P)]$$

Das engere Perzentilband (0.5 %–99.5 %) auf der Preisachse reflektiert die Tatsache, dass Preisveränderungen in illiquiden OTC-Märkten extremer ausfallen als Volumenabweichungen und daher eine aggressivere Bereinigung erfordern.

5.12 Off-Book-Anteil: Statistische Korrektur für Querschnittsanalysen

Der Off-Book-Anteil (`off_book_pct`) weist eine bemerkenswerte statistische Eigenschaft auf: an $> 99\%$ der Handelstage beträgt er für sämtliche ISINs exakt 0 %. Ein Einzeltages-Snapshot besitzt folglich **Nullvarianz**, was in der Korrelationsmatrix zu NaN-Werten führen würde.

Historisches-Mittel-Patching für Querschnittssignal

Anstelle des aktuellen Tageswerts wird der **historische Mittelwert** pro ISIN eingesetzt:

$$\text{off_book_pct}_i^{\text{latest}} \leftarrow \bar{x}_i = \frac{1}{T_i} \sum_{t=1}^{T_i} \text{off_book_pct}_{i,t}$$

Dies bewahrt das *Querschnittssignal* — also die Frage, welche Titel proportional mehr Off-Book-Handel aufweisen als andere — und eliminiert gleichzeitig die artifizielle Nullvarianz.

Kapitel 6

Dashboard-Architektur und Visualisierung

Ein Bild sagt mehr als tausend Zahlen — vorausgesetzt, es ist das richtige Bild.

6.1 Designphilosophie

Das OTC-X-Dashboard verkörpert drei Prinzipien, die der Schweizer Tradition institutionellen Designs entstammen:

1. **Klarheit vor Ornament** — jedes visuelle Element dient einem analytischen Zweck; dekorative Komponenten fehlen bewusst.
2. **Progressive Offenlegung** — Informationen werden über vier Tabs geschichtet, vom hochverdichteten KPI-Überblick bis zum granularen Einzeltitel-Deep-Dive.
3. **Kontextuelle Intelligenz** — Tooltips, Formeln und Hilfshinweise begleiten die Charts, sodass Nutzer stets verstehen, *was* sie sehen und *warum* es relevant ist.

6.2 Tab-Architektur

Tabelle 6.1: Dashboard-Tab-Struktur und Chart-Inventar

#	Tab	Inhalt	Charts
1	Overview	KPI-Karten (5), 90-Tage-Marktaktivität, Sektor-Treemap, Top Movers, Volumenverteilung	4
2	Market Data	26-Spalten-Explorer, Suche/Sortierung/Export, Security Deep-Dive mit σ -Band	1
3	Analytics	Korrelations-Heatmap, Volumen-Preis-Scatter, Amihud-Boxplots, Volatilitätstrend, 3D-Explorer	5
4	Anomaly Monitor	Severity-Karten, Anomalie-Treemap, Alert-Tabelle mit verlinkten ISINs	1

6.3 11 Charttypen im Überblick

Tabelle 6.2: Vollständiges Chart-Inventar — 11 interaktive Visualisierungstypen

#	Chart	Typ	Analytischer Zweck
1	Market Activity	Dual-Axis	Volumen-Balken + Trade-Count-Linie (90 Tage)
2	Sector Treemap	Treemap	Allokation nach Volumen, Farbe: $\emptyset$ Preisänderung
3	Top Movers	Bar	Grösste Gewinner/Verlierer nach $\Delta P\%$
4	Volume by Sector	Horizontal Bar	Sektorvolumen sortiert nach CHF-Total
5	Security History	Subplot	Preis $\pm \sigma$ -Band + Volumenbalken je ISIN
6	Correlation Matrix	Heatmap	Unterdreiecks-Pearson-Korrelation aller Metriken
7	Volume vs. Price	Scatter	Log-Skala $V \times \Delta P$, sektorkodiert
8	Amihud by Sector	Box Plot	λ -Verteilung pro Sektor
9	Volatility Trend	Line	Rolling σ mit SMA-/EWMA-Glättung
10	Anomaly Treemap	Treemap	Hierarchische Severity-Visualisierung
11	3D Explorer	3D Scatter	5-dimensionale interaktive Exploration

6.4 Volatilitätsglättung: SMA und EWMA

Das Dashboard bietet zwei Darstellungsmodi für die sektorale Volatilitätsentwicklung:

Dreistufige EWMA-Kaskade (Macro Smoothing)

$$\sigma_t^{(1)} = \text{SMA}_{90}(\sigma_{\text{daily}}) \quad (6.1)$$

$$\sigma_t^{(2)} = \text{EWM}_{120}(\sigma_t^{(1)}) \quad (6.2)$$

$$\sigma_t^{(3)} = \text{EWM}_{60}(\sigma_t^{(2)}) \quad (6.3)$$

Die resultierende Serie $\sigma_t^{(3)}$ eliminiert kurzfristige Rauscheffekte und hebt strukturelle Volatilitätsregime hervor.

6.5 Dashboard-Screenshots

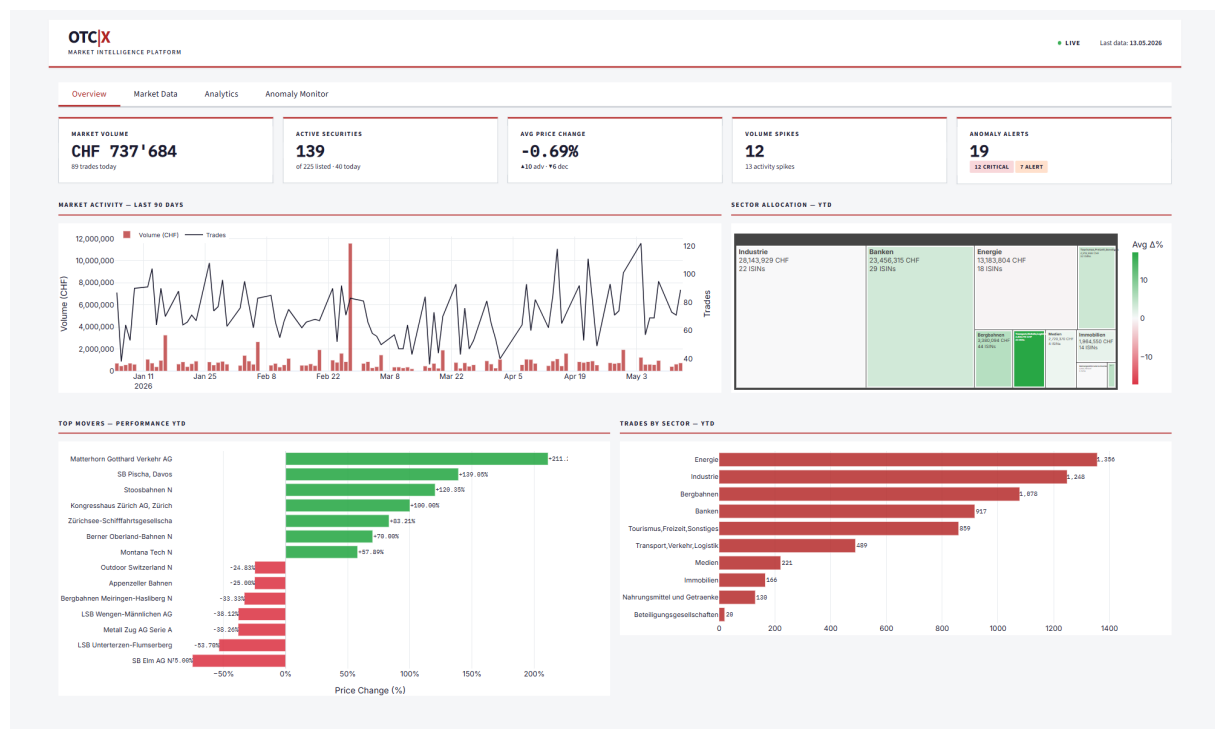


Abbildung 6.1: Tab 1: Overview — KPI-Karten, Marktaktivität, Sektor-Treemap

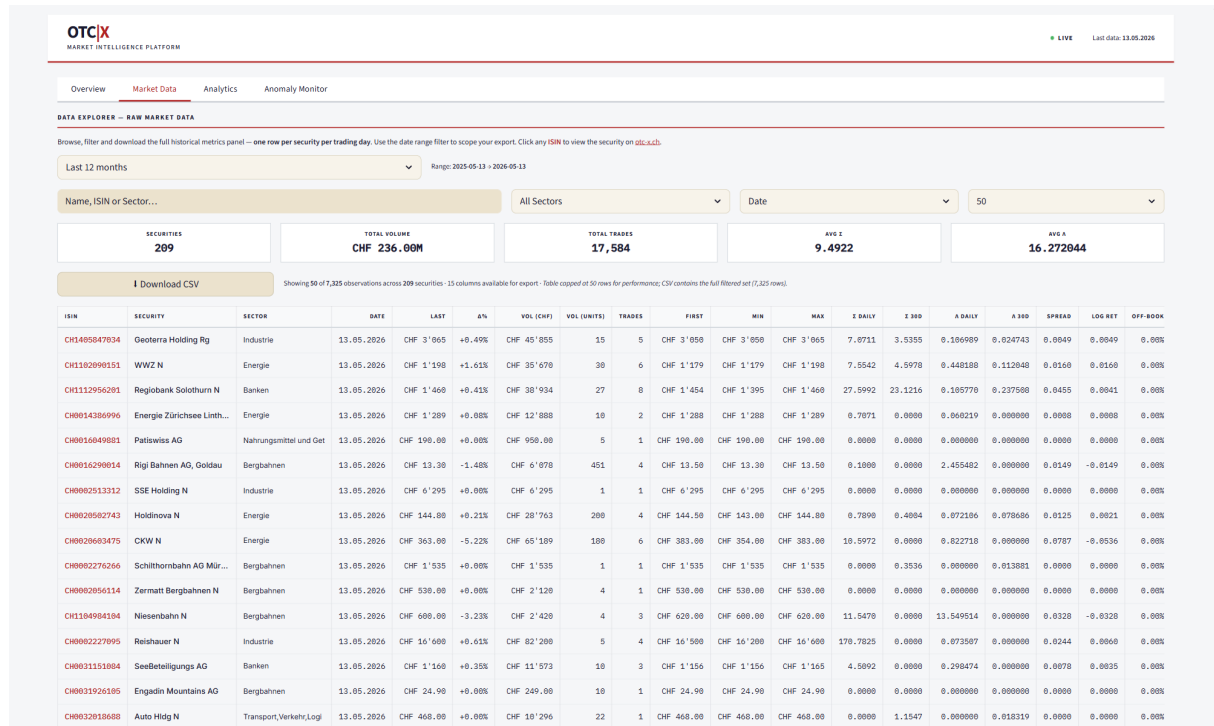


Abbildung 6.2: Tab 2: Market Data — 26-Spalten-Explorer mit Suche und Export

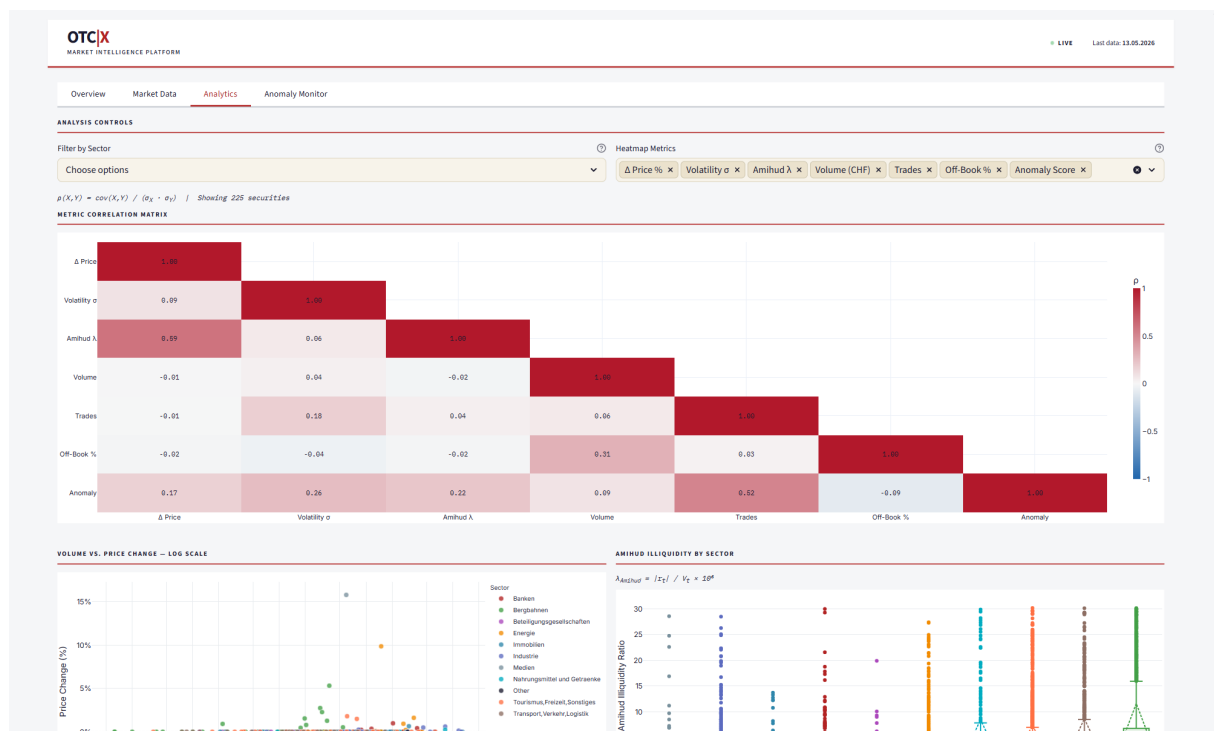


Abbildung 6.3: Tab 3: Analytics — Korrelations-Heatmap, Amihud-Boxplots, 3D-Explorer

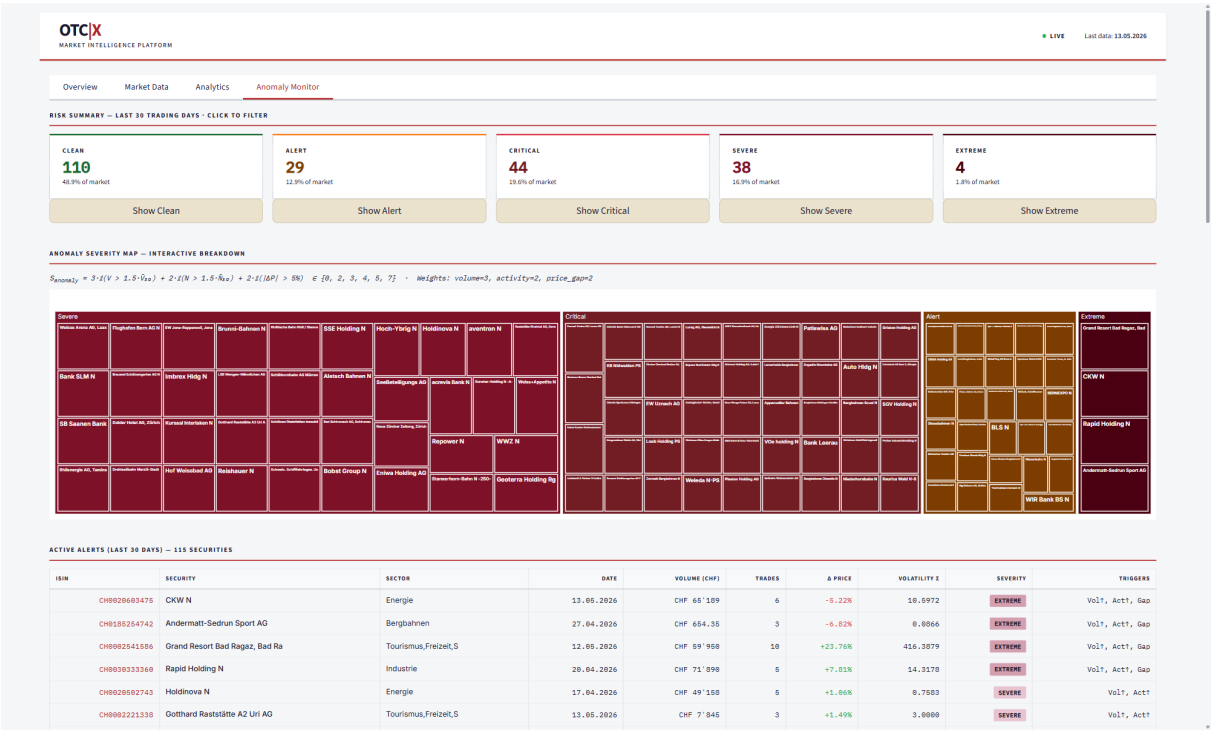


Abbildung 6.4: Tab 4: Anomaly Monitor — Severity-Karten und Alert-Tabelle

Kapitel 7

Qualitätssicherung und Engineering-Standards

Vertrauen entsteht nicht durch Versprechen, sondern durch Verifikation.

7.1 Teststrategie

Die Codebasis wird durch **115 automatisierte Tests** in fünf Kategorien abgesichert:

Tabelle 7.1: Teststruktur nach Modul und Kategorie

Testmodul	Kategorie	Abdeckung
test_backend.py	Unit + Integration	ISIN-Berechnung (Luhn), Metriken-Engine, Pipeline-Imports
test_frontend.py	Unit + Smoke	Formatierungs-Utilities (CHF, %, Badge), Config, Chart-Instantiierung
test_paths.py	Integration	Pfadauflösung, Datei-Existenzprüfung über alle Module
test_integration.py	Integration	Metriken-Verkettung, Data-Loader-Integration
test_acceptance.py	BDD-Akzeptanz	Geschäftslogik-Validierung via pytest-bdd

7.2 Code-Konventionen

- **Type Hints:** Sämtliche Funktionssignaturen tragen vollständige Typannotationen (PEP 484).
- **Docstrings:** NumPy-Style mit Parameters, Returns, Raises und Notes-Sektionen.
- **Pfadauflösung:** Ausschliesslich via `Path(__file__).resolve().parent`-Ketten; keine hardcodierten Pfade.

- **Zahlenformatierung:** Schweizer Konventionen (Apostroph als Tausendertrenner, CHF-Währung).
- **Backend/Frontend-Separation:** Backend nutzt Polars; Frontend nutzt Pandas. Keine Vermischung.

7.3 Pfadauflösungsstrategie

Listing 7.1: Pfadauflösungsmuster — keine hartcodierten Pfade

```
1 from pathlib import Path
2
3 # Backend: ein Parent hoch von operations/ zu backend/
4 script_dir = Path(__file__).resolve().parent
5 data_dir   = script_dir.parent / "data"
6
7 # Frontend: drei Parents hoch zur Wurzel, dann in backend/data/
8 root      = Path(__file__).resolve().parent.parent.parent
9 data_dir  = root / "backend" / "data"
```

Kapitel 8

Glossar und Referenzen

8.1 Fachbegriffe

Begriff	Definition
Amihud-Ratio (λ)	Mass für den Preisimpakt pro gehandelte Volumeneinheit; hohe Werte indizieren Illiquidität
Anomaly Score	Gewichteter Composite-Wert ($S \in \{0, 2, 3, 4, 5, 7\}$) aus drei binären Flags
Corwin–Schultz Proxy	Approximation der Geld-Brief-Spanne via logarithmisches Hoch-Tief-Verhältnis
EWMA	Exponentially Weighted Moving Average; gewichtete Glättung mit Vergangenheitsdämpfung
IREB	International Requirements Engineering Board; Zertifizierungsstandard für RE
ISIN	International Securities Identification Number; 12-stelliger alphanumerischer Code
Luhn-Algorithmus	Prüfzifferverfahren nach ISO 7812 zur Validierung von Identifikationsnummern
OTC-X	Ausserbörsliche Handelsplattform der Berner Kantonalbank für Schweizer Titel
Parquet	Spaltenorientiertes Speicherformat für analytische Workloads (Apache Foundation)
Polars	Hochperformante DataFrame-Engine in Rust mit Python-Bindings
Rolling Median	Gleitender Median über ein definiertes Zeitfenster; robust gegenüber Ausreissern
Severity-Tier	Klassifikationsstufe (Clean bis Extreme) basierend auf dem Composite Anomaly Score
Valor	Schweizerische Wertpapieridentifikationsnummer (6–7 Ziffern)

8.2 Literatur und Standards

1. Amihud, Y. (2002). *Illiquidity and stock returns: cross-section and time-series effects*. Journal of Financial Markets, 5(1), 31–56.
2. Corwin, S. A. & Schultz, P. (2012). *A simple way to estimate bid-ask spreads from daily high*

- and low prices*. The Journal of Finance, 67(2), 719–760.
3. IREB (2023). *Certified Professional for Requirements Engineering – Foundation Level Syllabus*, Version 3.1.2. International Requirements Engineering Board.
 4. ISO 6166:2021. *Financial services – International securities identification number (ISIN)*.
 5. ISO 7812-1:2017. *Identification cards – Identification of issuers – Part 1: Numbering system* (Luhn).
 6. ISO 9241-210:2019. *Ergonomics of human-system interaction – Part 210: Human-centred design*.
 7. McKinney, W. (2017). *Python for Data Analysis*, 2nd ed. O'Reilly Media.
 8. Ritchie, J. et al. (2024). *Polars: Blazingly fast DataFrames in Rust and Python*. <https://pola.rs>

Kapitel A

Output-Schema-Referenz

Tabelle A.1: Vollständiges Schema von `daily_metrics.parquet` (26 Spalten)

#	Spalte	Typ	Beschreibung
1	Isin	String	International Securities Identification Number
2	Name	String	Unternehmensname (aus Wertpapier-Metadaten)
3	Sektor	String	Wirtschaftliche Sektorklassifikation
4	Datum	Date	Handelstag
5	trades_today	UInt32	Anzahl Trades an diesem Tag
6	volume_today_units	Float64	Gesamtvolumen in Stück
7	volume_today_chf	Float64	Gesamtvolumen in CHF
8	price_min	Float64	Intraday-Tiefstkurs
9	price_max	Float64	Intraday-Höchstkurs
10	price_first	Float64	Eröffnungskurs (erster Trade)
11	price_last	Float64	Schlusskurs (letzter Trade)
12	price_change_pct	Float64	Tägliche Preisveränderung (%)
13	log_returns	Float64	Natürlicher Logarithmus des Kursverhältnisses
14	volatility_daily	Float64	Intraday-Standardabweichung der Kurse
15	amihud_daily	Float64	Amihud-Illiquiditätsratio (λ)
16	spread_log_hl	Float64	Logarithmischer High-Low Spread Proxy
17	trade_duration_min	Float64	Maximale Lücke zwischen konsekutiven Trades (Min.)

Tabelle A.1: (Fortsetzung)

#	Spalte	Typ	Beschreibung
18	off_book_pct	Float64	Anteil Off-Book-Trades (%)
19	trades_30d_median	Float64	30-Handelstage-Rolling-Median der Trade-Anzahl
20	volume_30d_median	Float64	30-Handelstage-Rolling-Median des CHF-Volumens
21	volatility_30d_median	Float64	30-Handelstage-Rolling-Median der Volatilität
22	amihud_30d_median	Float64	30-Handelstage-Rolling-Median des Amihud λ
23	volume_spike	Boolean	Flag: $V > 1,5 \times \tilde{V}_{30}$
24	activity_spike	Boolean	Flag: $N > 1,5 \times \tilde{N}_{30}$
25	price_gap	Boolean	Flag: $ \Delta P\% > 5$
26	anomaly_score	UInt8	Composite Anomaly Score $\in [0, 7]$

Kapitel B

Abkürzungsverzeichnis

API Application Programming Interface.

BEKB Berner Kantonalbank AG.

CD Continuous Deployment.

CI Continuous Integration.

ETL Extract, Transform, Load.

IREB International Requirements Engineering Board.

ISIN International Securities Identification Number.

OTC | X · Liquidity Intelligence Radar · Swiss OTC Market

Dieses Dokument ist Bestandteil des proprietären OTC-X Projekts.

© 2026 Mihael Atanasov · mihaelatanasov22@gmail.com · Alle Rechte vorbehalten.

Erstellt für die Berner Kantonalbank AG, Schwarzenburgstrasse 160, 3001 Bern.

Mit Präzision gefertigt • für den Schweizer OTC-Markt
